

Title

- ▶ Synchronous & Asynchronous Programming
- ▶ Threads, Tasks & File Transfer in ASP.NET Core

By MUHYDEEN ADEDIRAN

For Zentrium Limited

Agenda

- ▶ • Synchronous vs Asynchronous
- ▶ • Threading Fundamentals
- ▶ • Task-based Asynchronous Pattern (TAP)
- ▶ • `async` & `await`
- ▶ • Performance & Scalability
- ▶ • File Handling in .NET
- ▶ • File Upload & Download in ASP.NET Core
- ▶ • Streaming & Large File Transfer
- ▶ • Best Practices & Pitfalls

Synchronous Programming

- ▶ Synchronous execution runs tasks one after another.
- ▶ Each operation blocks the thread until completion.

Synchronous Example

```
▶ public string ReadFile()  
▶ {  
▶     return File.ReadAllText("data.txt");  
▶ }
```

Problems with Sync Code

- ▶ • Blocks threads
- ▶ • Poor scalability
- ▶ • API freezing
- ▶ • Inefficient resource usage

Asynchronous Programming

- ▶ Asynchronous programming allows tasks to run without blocking the main thread.
- ▶ Improves responsiveness and scalability.

Async Example

```
▶ public async Task<string> ReadFileAsync()  
▶ {  
▶     return await File.ReadAllTextAsync("data.txt");  
▶ }
```

Thread Basics

- ▶ A thread is the smallest unit of execution.
- ▶ • Main Thread
- ▶ • Worker Threads
- ▶ • Thread Pool

Creating Threads

- ▶ Thread thread = new Thread(() =>
- ▶ {
- ▶ Console.WriteLine("Running");
- ▶ });
- ▶ thread.Start();

Thread Pool

- ▶ `ThreadPool.QueueUserWorkItem(_ =>`
- ▶ `{`
- ▶ `Console.WriteLine("Thread pool work");`
- ▶ `});`

Task-Based Pattern

- ▶ Task represents an asynchronous operation built on the thread pool.

Task Example

```
▶ public async Task DoWork()  
▶ {  
▶     await Task.Delay(1000);  
▶     Console.WriteLine("Done");  
▶ }
```

Task vs Thread

- ▶ Thread = low-level control
- ▶ Task = high-level & scalable

async and await

- ▶ async enables awaiting
- ▶ await pauses without blocking thread

Execution Flow

- ▶ 1. Method starts
- ▶ 2. Hits await
- ▶ 3. Thread released
- ▶ 4. Resumes when done

Parallel Tasks

▶ `await Task.WhenAll(task1, task2);`

When to Use Async

- ▶ • I/O operations
- ▶ • Database calls
- ▶ • API calls
- ▶ • File access

CPU Bound Parallel

▶ `Parallel.For(0, 5, i => Console.WriteLine(i));`

File Handling

- ▶ System.IO namespace provides file operations.

Write File

▶ `File.WriteAllText("log.txt", "Hello World");`

Async Write

▶ `await File.WriteAllTextAsync("log.txt", "Hello");`

File Upload ASP.NET Core

- ▶ IFormFile represents uploaded files.

Upload Example

- ▶ [HttpPost("upload")]
- ▶ public async Task<ActionResult> Upload(IFormFile file)
- ▶ {
- ▶ var path = Path.Combine("uploads", file.FileName);
- ▶ using var stream = new FileStream(path, FileMode.Create);
- ▶ await file.CopyToAsync(stream);
- ▶ return Ok();
- ▶ }

Download Example

- ▶ [HttpGet("download/{name}")]
- ▶ public IActionResult Download(string name)
- ▶ {
- ▶ var bytes = System.IO.File.ReadAllBytes(name);
- ▶ return File(bytes, "application/octet-stream", name);
- ▶ }

Streaming Large Files

▶ `return File(stream, "application/octet-stream");`

File Transfer Flow

▶ Client → Request → Server → Storage → Response

Security

- ▶ • Validate file type
- ▶ • Limit size
- ▶ • Scan files
- ▶ • Store outside web root

Performance Tips

- ▶ • Use async for I/O
- ▶ • Stream large files
- ▶ • Avoid blocking calls

Common Pitfalls

- ▶ • Using `.Result` on `async`
- ▶ • Forgetting `await`
- ▶ • Thread exhaustion

Conclusion

- ▶ Async programming and efficient file transfer enable scalable and responsive ASP.NET Core applications.

Async Execution Flow Diagram

Request

Await I/O

Response